OBJECT-ORIENTED PROGRAMMING (OOP)

Lecture 2 — Detailed Study Notes

Programming history • Why OOP • Objects and classes • Abstraction • Encapsulation

Coverage note: These notes follow the supplied lecture transcript. The lecture explains programming history, the motivation for OOP, the car/owner example, abstraction, and encapsulation. The teacher says inheritance and polymorphism will be covered in the next part, so they are not treated here as topics taught in this lecture.

Designed for study, revision, and uploading to a GitHub notes repository.

1. Lecture overview

This lecture builds the foundation for Low-Level Design (LLD) by explaining Object-Oriented Programming (OOP). The central idea is to model software in a way that resembles the real world: a program contains objects, each object has characteristics and behaviours, and objects interact with one another.

- Understand why programming evolved from machine code to assembly, procedural programming, and OOP.
- Understand the limitations of procedural programming for complex applications.
- Distinguish an object, its characteristics, its behaviours, and a class.
- Understand why classes make real-world modelling easier than a collection of unrelated variables and functions.
- Learn abstraction: hide unnecessary implementation details and expose the interface needed by a user of the object.
- Learn encapsulation: keep data and related behaviours together, and control access to sensitive state.
- Understand public, private, and protected access modifiers, plus getters and setters.

Prerequisite stated by the teacher: some familiarity with C++ classes and objects can help. The teacher recommends a more beginner-oriented OOP playlist for learners who have never encountered classes or objects.

2. History of programming

The teacher introduces programming history to explain why OOP became useful. Each stage made programming more convenient, but earlier approaches had limitations when programs became large and complex.

2.1 Machine language

Machine language consists of binary instructions—zeros and ones—that the CPU can execute directly.

- Example form: a sequence such as 01010110101. The transcript uses this only as an illustrative binary sequence.
- Advantage: it is directly executable by the processor.
- Limitations: it is difficult for humans to read and write, mistakes are easy to make, a small change can alter the meaning of an instruction, and writing large applications this way is impractical.

2.2 Assembly language

Assembly language introduced mnemonics—short symbolic instruction names such as **MOV**—instead of requiring programmers to write only binary.

- The transcript gives an illustrative instruction similar to **MOV A, 61H**, described as moving a hexadecimal value into a register.
- This is more readable than raw binary, but assembly remains closely tied to the processor architecture and hardware details.
- Programmers must still manage low-level instructions and registers, making mistakes possible and large-scale development difficult.

2.3 Procedural programming

Procedural programming introduces familiar programming constructs such as functions, loops, conditional blocks, if/else statements, and switch statements. A program is organised as a sequence of instructions or procedures.

- The teacher names C as an example of a procedural programming language.
- It works well for many smaller tasks and makes programs more understandable than raw machine code or assembly.
- The lecture argues that, as applications become more complex, procedural code can become difficult to organise, extend, reuse, and map to real-world entities.

2.4 Object-Oriented Programming

OOP introduces a way to organise data and related behaviours around objects. Rather than representing a real-world entity through many separate variables and functions, the program can model the entity as an object with its own state and operations.

Stage	Main idea	Limitation highlighted in the lecture
Machine language	Binary instructions executed directly by the CPU.	Hard to read/write; error-prone; unsuitable for large applications.
Assembly language	Human-readable mnemonics for low-level instructions.	Still hardware-dependent and low-level; complex to scale.
Procedural programming	Functions, loops, conditions, and instruction sequences.	Real-world modelling, data protection, reuse, and large-scale organisation become difficult.
OOP	Objects combine characteristics and behaviours and interact with other objects.	Requires learning concepts such as classes, abstraction, encapsulation, and other OOP pillars.

3. Why OOP matters

The lecture gives three major reasons for moving beyond procedural programming:

- **Real-world modelling:** real systems contain identifiable entities that interact. OOP lets code represent these entities and their interactions more directly.
- **Data security:** some information should not be freely changed by any outside part of a program. OOP provides mechanisms such as access control to protect an object's state.
- **Scalability and reusability:** grouping data and behaviour into well-defined objects can make complex applications easier to extend and reuse.

Examples of real-world systems named by the teacher include clones of Uber, Ola, Zomato, and Paytm. Such applications contain users, vehicles or delivery partners, orders, payments, and other entities that interact.

> **Important framing:** The lecture presents OOP as a way to organise code for complex real-world problems. It does not claim that procedural programming cannot solve any complex task; rather, the focus is on the organisational difficulties that arise as a system grows.

4. Objects: characteristics and behaviours

The teacher describes a real-world object as having two broad aspects:

Aspect	Meaning	Car example
Characteristics (state/data)	Information or properties that describe an object.	Brand, model, engine-on status, wheels, current speed, current gear.
Behaviours (actions/methods)	Operations the object can perform.	Start engine, stop engine, shift gear, accelerate, brake.

In programming, characteristics are commonly represented by data members or variables. Behaviours are represented by functions associated with the object, often called methods.

4.1 What is a class?

A **class** is a blueprint or template for creating objects. It describes what data and behaviours objects of that type will have. The lecture uses a **Car** class as the blueprint for cars.

- The class describes the shape and capabilities of a car object.
- Individual objects can be created from the class, each with its own values—for example, different brands and models.
- The teacher illustrates object creation with C++-style syntax such as `Car myCar = new Car`. Exact syntax depends on the language and class setup.

4.2 What is an object?

An **object** is a particular instance created from a class. If `Car` is the blueprint, `myCar` can refer to one particular car object with a specific brand, model, speed, and gear.

> **Quick memory aid:** Class = blueprint; object = instance. Characteristics = what the object knows or stores; behaviours = what the object can do.

5. Procedural programming vs OOP: the car and owner example

The teacher uses a simple relationship: an owner owns and drives a car. Both the owner and the car are real-world objects, and the owner interacts with the car.

5.1 Modelling the car procedurally

Without classes, the program might represent a car using separate variables such as brand, model, and whether the engine is on. Functions such as start, stop, and shift gear are also defined separately.

- A single car may be manageable, but a second car can require another set of variables: brand2, model2, engineOn2, and so on.
- More owners create more variables and potentially different functions or logic for each owner.
- The relationship between a particular owner and a particular car is not clearly represented by the collection of variables and function calls.
- To model driving, a drive function may need to receive the car's brand and model and then call the car-related functions in the right sequence.

The lecture uses this to show how a simple real-world relationship can become a collection of disconnected data and procedural steps.

5.2 Modelling the same scenario with OOP

With OOP, the program can define a `Car` class and an `Owner` class.

- The Car class contains car characteristics and methods such as start, shift gear, accelerate, brake, and stop.
- The Owner class can contain the owner's name and a car object representing the car they own.
- The Owner's drive method can interact with its car object directly, for example by calling the car's start or shift-gear behaviour.
- The owner-to-car relationship is expressed in the object structure rather than being inferred from a loose collection of variables.

Procedural approach	OOP approach
Car details may be scattered across separate variables.	Car details are grouped inside a Car class/object.
Functions may need car details passed around explicitly.	Methods can work with the state of the relevant object.
Multiple cars/owners can cause repeated variables and branching logic.	Multiple instances can be created from reusable class definitions.
Relationships between entities are less explicit.	An Owner object can hold or refer to a Car object.

The teacher's point is not that procedural code can never represent this scenario; it is that OOP offers a more direct way to express entities and their interactions.

6. The ideology of OOP

The lecture summarises OOP's basic philosophy as: **make programming work more like the real world being modelled.**

- Represent important real-world entities as objects.
- Give each object the characteristics and behaviours relevant to it.
- Let objects interact through methods or by being passed to other objects.
- When reading OOP code, look for the objects and ask how they communicate. One object may call another object's method, or an object may be supplied as a parameter.

This way of thinking helps a developer reason about a system as cooperating objects instead of viewing the entire program only as a long sequence of instructions.

7. Pillars of OOP — lecture scope

The teacher introduces the four pillars of OOP, but this transcript develops the first two in detail: abstraction and encapsulation. Inheritance and polymorphism are explicitly deferred to the next part of the series.

Pillar	Coverage in this lecture
Abstraction	Explained in detail with the car interface and the sports-car implementation.
Encapsulation	Explained in detail with data grouping, private members, access modifiers, and getters/setters.
Inheritance	Mentioned in the abstraction code example; full explanation is deferred to the next lecture part.
Polymorphism	Named as a pillar, but the teacher says it will be covered in the next part.

8. Abstraction

8.1 Intuitive, real-world meaning

Abstraction means hiding unnecessary internal details from the person or object using something and exposing only what is needed to use it.

The teacher uses three everyday examples:

- **Car:** a driver uses controls such as the ignition, clutch, gear shifter, accelerator, and brake. The driver does not need to understand every detail of how the engine or gearbox works to drive the car.
- **Laptop or mobile:** a user watches a video through the screen without needing to know the internal hardware design.
- **Television:** the viewer uses a remote control without needing to understand the internal wiring and broadcast circuitry.

8.2 Definition to remember

> **Abstraction:** Hide unnecessary details from a client and expose only what is necessary for that client to use the object.

Here, a **client** means the code or user that interacts with an object. It needs to know which operations are available and how to call them, but it generally does not need to know how those operations are implemented internally.

8.3 Interface vs implementation

- **Interface:** the available operations through which another part of the program interacts with an object.
- **Implementation:** the internal code that performs those operations.

In the car example, methods such as `startEngine()`, `shiftGear()`, `accelerate()`, `brake()`, and `stopEngine()` form the usable interface. The caller can use these methods without knowing their internal steps.

8.4 Abstract Car and SportsCar example

The transcript describes a C++-style design in which a base `Car` class declares the operations a car should provide. The methods are described as virtual and are not implemented in the base class; a child class such as `SportsCar` supplies the concrete behaviour.

- The abstract Car class acts as a contract or interface: it names the behaviours available to a caller.
- The SportsCar class implements those behaviours and stores specific state such as brand, model, engine-on status, current speed, and current gear.
- The caller creates a SportsCar object and uses the exposed methods, without needing to inspect the implementation of each method.
- The transcript uses a Ford Mustang as an illustrative sports-car object.

8.5 Example behaviour described in the lecture

- **Start engine:** marks the engine as on and prints a message such as the engine starting with a roar.
- **Shift gear:** checks whether the engine is on. If it is off, gear shifting is rejected; otherwise, the current gear is updated.
- **Accelerate:** checks that the engine is on, increases the speed in the illustrative implementation, and prints the new speed.
- **Brake:** decreases speed and prevents the speed from going below zero.
- **Stop engine:** turns the engine off and resets the example's speed and gear state.

The transcript's code example is illustrative rather than a complete vehicle simulation. The key abstraction lesson is that a caller can use the methods without knowing the internal state changes.

8.6 Abstraction beyond classes

The teacher also describes high-level programming languages as an example of abstraction. A programmer can write an `if` statement or a `for` loop without writing machine instructions directly. The compiler translates the high-level program into lower-level instructions.

> **Abstraction takeaway:** Focus on what an object offers and how to use it, not on every internal detail of how it works.

9. Encapsulation

9.1 The capsule analogy

Encapsulation combines an object's data and its related methods inside one unit. The teacher compares this to medicine inside a capsule: the related contents are held together within one boundary.

- A Car object contains car state, such as brand, model, engine status, speed, and gear.
- It also contains behaviours that operate on that state, such as starting, accelerating, braking, and stopping.
- In programming, the class is the structure used to group these related data members and methods.

9.2 Encapsulation also controls access

The teacher emphasises that encapsulation is not only grouping data and behaviour. It also helps protect important state from being changed arbitrarily by outside code.

The car examples include the odometer reading and current speed. A user should not be able to directly set an arbitrary odometer value or force the car's speed to 500 km/h by changing an internal variable. Instead, state should change through controlled behaviours and rules.

9.3 Abstraction vs encapsulation

Abstraction	Encapsulation
Focuses on hiding unnecessary implementation details and exposing the operations needed by the client.	Focuses on grouping related data and methods, and controlling access to the object's internal state.
Question: What does the client need to know or use?	Question: What belongs together, and who is allowed to access or modify it?
Example: a driver uses the car controls without understanding the engine internals.	Example: currentSpeed is private and can only change through controlled methods.

The lecture contrasts abstraction's information hiding with encapsulation's data protection. They are related concepts and can work together in the same class.

9.4 Access modifiers in C++

The teacher introduces three access modifiers: public, private, and protected.

Modifier	Meaning in the lecture	Typical use
public	Accessible from outside the class through the object's public interface.	Methods intended for callers, such as startEngine() or getCurrentSpeed().
private	Not directly accessible from outside the class.	Internal state such as currentSpeed, currentGear, or engineOn when direct modification must be controlled.
protected	Not generally accessible from unrelated outside code, but accessible to derived/child classes under C++ access rules.	State or methods intended for use by a class hierarchy; the lecture says this will become clearer when inheritance is studied.

9.5 Why public data can be dangerous

In the example, if current speed is public, outside code could assign an unrealistic value directly:

```
mySportsCar.currentSpeed = 500;
```

The lecture uses this to show that unrestricted access can violate the object's rules. Declaring internal variables private prevents this kind of direct assignment from ordinary outside code.

9.6 Getters and setters

Once a variable is private, outside code cannot read or change it directly. A class can selectively expose public methods called getters and setters.

- **Getter:** returns a value, such as `getCurrentSpeed()`. The caller can inspect the speed without directly accessing the private variable.
- **Setter:** changes a value, such as `setTyre(...)`. The method can validate the new value before updating internal state.
- A class does not need to provide both a getter and a setter for every private field. For example, the lecture suggests allowing current speed to be read while not exposing a direct setter.
- A tyre/company field may have both a getter and a setter, but the setter can check that the supplied tyre name is valid or supported before accepting it.

The key benefit is control: validation and business rules can be placed in one location instead of allowing every caller to change state freely.

9.7 Encapsulation summary

- Keep related state and behaviour together in a class.
- Use access modifiers to control visibility.
- Make internal data private when it must not be changed directly.
- Expose only the public methods needed by callers.
- Use getters to provide controlled read access and setters when controlled write access is appropriate.
- Put validation inside setter methods or other domain methods when a state change must obey rules.

Important nuance: A getter or setter is not automatically secure just because it exists. The benefit comes from controlling access and applying the appropriate rules. A setter that accepts every value without validation may still allow invalid state.

10. How abstraction and encapsulation work together

A well-designed Car class can use both ideas at once:

- Abstraction: callers see a small, clear set of operations—start engine, shift gear, accelerate, brake, stop engine—rather than the full implementation.
- Encapsulation: the class keeps state such as current speed and current gear private and changes it through controlled methods.

For example, a caller can request acceleration without directly editing currentSpeed. The class decides how speed changes and whether acceleration is allowed when the engine is off.

Scenario	Concept illustrated	Reason
A caller can use accelerate() without seeing its implementation.	Abstraction	Internal mechanics are not required for using the operation.
currentSpeed is private and cannot be directly assigned from outside.	Encapsulation / access control	Prevents uncontrolled state changes.
setTyre() validates the requested tyre value.	Encapsulation / controlled mutation	Centralises rules for valid state.
Car groups speed, gear, engine state, and related methods.	Encapsulation	Related data and behaviours are kept together.

11. Key terms for revision

Term	Meaning
Machine language	Binary instructions executed directly by the CPU.
Assembly language	Low-level programming using mnemonics such as MOV.
Procedural programming	Programming organised around procedures/functions and sequences of instructions.
OOP	Programming organised around objects that contain state and behaviours and interact.
Object	An instance representing a particular entity with state and behaviour.
Class	A blueprint defining the data and behaviours of objects of that type.
Characteristic / state	Data describing the current properties of an object.
Behaviour / method	An operation an object can perform.
Abstraction	Hide unnecessary details and expose the interface needed by a client.
Encapsulation	Group related data and methods and control access to internal state.
Interface	The operations exposed to another part of the program.
Implementation	The internal code that performs an operation.
Access modifier	A language feature controlling where class members can be accessed.
Getter	A method that returns a value from an object.
Setter	A method that changes a value, potentially after validation.

12. Practice questions

- **1.** Why does the lecture begin OOP with the history of programming?
- **2.** What is machine language, and what limitations does the teacher highlight?
- **3.** What are mnemonics in assembly language? Why is assembly still low-level?
- **4.** Which programming constructs does the lecture associate with procedural programming?
- **5.** What three reasons for using OOP are emphasised?
- **6.** Explain the difference between a class and an object using the car example.
- **7.** List five characteristics and five behaviours that a Car object might have.
- **8.** Why does modelling an owner and multiple cars become awkward in the procedural example?
- **9.** State the basic ideology of OOP in one or two sentences.
- **10.** Define abstraction using the car, laptop, or television example.
- **11.** What is the difference between an interface and an implementation?
- **12.** How does the abstract Car / SportsCar example demonstrate abstraction?
- **13.** Define encapsulation using the capsule analogy.
- **14.** Why should currentSpeed or odometer state not be freely modifiable from outside?
- **15.** Explain public, private, and protected access modifiers.
- **16.** What are getters and setters? Why might a setter validate input?
- **17.** Give one example each of abstraction and encapsulation in the same Car class.
- **18.** Which two OOP pillars are covered in this lecture, and which are deferred?

13. Short self-check answers

Programming history: It explains the limitations of earlier approaches and why a way to model complex real-world entities became useful.

Machine language: Binary instructions executed by the CPU; difficult to read/write, error-prone, and impractical for large applications.

Assembly: Mnemonics such as MOV make instructions more readable, but the code remains tied to hardware details.

Procedural constructs: Functions, loops, if/else blocks, switch statements, and sequences of instructions.

Reasons for OOP: Real-world modelling, data protection, and improved organisation/reuse for complex applications.

Class vs object: Car is a blueprint; myCar is one instance with particular values.

Car examples: Characteristics: brand, model, engineOn, currentSpeed, currentGear. Behaviours: start, stop, shiftGear, accelerate, brake.

Procedural modelling issue: Data and behaviour can become scattered, repeated for each car/owner, and relationships are less explicit.

OOP ideology: Represent entities as objects and let those objects interact through their behaviours and interfaces.

Abstraction: Expose only the operations a user needs while hiding unnecessary internal details.

Interface vs implementation: The interface describes how to use a component; the implementation is the internal code that makes it work.

Abstract Car example: The base class declares common car operations; SportsCar implements them; the caller uses the operations without inspecting their internals.

Encapsulation: Keep an object's related data and methods together and control access to its internal state.

Private state: Uncontrolled edits can create invalid or unrealistic values and break the object's rules.

Access modifiers: public: accessible through the class interface; private: class-internal; protected: available to the class and derived classes according to language rules.

Getters/setters: A getter reads a value; a setter changes a value and can validate it before accepting it.

Both concepts: Call accelerate() without knowing its internals (abstraction); keep currentSpeed private and change it only through allowed methods (encapsulation).

Lecture scope: Abstraction and encapsulation are explained; inheritance and polymorphism are deferred to the next part.

14. Final lecture summary

The lecture argues that OOP helps developers model real-world entities more directly. A class acts as a blueprint; an object is an instance with characteristics and behaviours. Objects interact through methods and references to one another.

Abstraction exposes the operations needed by a client while hiding unnecessary implementation details. Encapsulation groups data and related behaviours and controls how internal state can be accessed or changed. In C++, access modifiers and carefully designed getters/setters are tools for implementing these ideas.

> **One-line revision:** Abstraction is about what to expose; encapsulation is about grouping state with behaviour and controlling access to that state.

Source note: Prepared from the supplied Hindi/Hinglish lecture transcript. Obvious technical terms have been standardised for readability. The notes preserve the lecture's examples and scope; explanatory phrasing is used to make the concepts easier to revise.